Homework 8

Q1 Apply KMeans algorithm to cluster the following four objects (with (x, y) representing locations) into two clusters. Initial cluster centers are: Medicine A (1, 1) and Medicine B (2, 1). Use Euclidean distance. Explain each of the clustering steps and the clustering result after each iteration.

Objects	Attribute X	Attribute Y
Medicine A	1	1
Medicine B	2	1
Medicine C	4	3
Medicine D	5	4

Centroid (c1) = (1, 1)
Centroid (c2) = (2, 1)

Calculating the distance from the centroid:

Objects	Distance from c1	Distance from c2
Medicine A	0	1
Medicine B	1	0
Medicine C	3.6	2.8
Medicine D	5	4.24

Rounding off the distance:

Objects	Distance from c1	Distance from c2	Cluster
Medicine A	0	1	c1
Medicine B	1	0	c2
Medicine C	4	3	c2
Medicine D	5	4	c2

Calculating the new centroid:
c1 (1, 1)
c2 (3.67, 2.67)

Calculating the distance from the centroid:

Objects	Distance from c1	Distance from c2
Medicine A	0	3.05
Medicine B	1	226
Medicine C	3.6	0.56
Medicine D	5	1.97

Rounding off the distance:

Objects	Distance from c1	Distance from c2	Cluster
Medicine A	0	3	c1
Medicine B	1	2	c1
Medicine C	4	1	c2
Medicine D	5	2	c2

Calculating the new centroid:
c1 (1.5, 1)
c2 (4.5, 3.5)

Calculating the distance from the centroid:

Objects	Distance from c1	Distance from c2
Medicine A	0.5	4.3
Medicine B	0.5	3.54
Medicine C	3.2	0.71
Medicine D	4.6	0.71

Rounding off the distance:

Objects	Distance from c1	Distance from c2	Cluster
Medicine A	1	4	c1
Medicine B	1	4	c1
Medicine C	3	1	c2
Medicine D	5	1	c2

Objects and Resulting Cluster

Objects	Attribute X	Attribute Y	Cluster
Medicine A	1	1	c1
Medicine B	2	1	c1
Medicine C	4	3	c2
Medicine D	5	4	c2

We can see that the values in the last two clusters remain the same so, we stop. Our final centroids are c1 (1.5, 1), c2 (4.5, 3.5) and c1 includes Medicine A and Medicine B and c2 includes Medicine C and Medicine D.

Q2. Choose an appropriate data set **other than Iris data set** and apply K-means to cluster the data using **R and Weka**. You should **use the same data set** and **set the same number of clusters** for both tools. Describe the clustering results generated from these two tools, compare them, and explain which result is better than the other.

a) R Clustering Implementation

```
6  kdata <- read.arff("cpu.arff")
7  View(kdata)
8
9  kdata.features = kdata
10 kdata.features$class <- NULL
11 View(kdata.features )
12
13 results <- kmeans(kdata.features, 3)
14 results
15
16 results$size
17 results$cluster
18 results$centers
19 results$totss
20 results$withinss
21 results$tot.withinss
22 results$betweenss
23 results$iter
24 results$ifault
25
26 table(kdata$class, results$cluster)
27 plot(kdata, col = results$cluster)
28
29 colnames(kdata)
30
31 plot(kdata[c("MYCT" , "MMIN" , "MMAX" , "CACH" , "CHMIN", "CHMAX" , "class")], col=results$cluster)
32 points(results$centers[,c("MYCT")], col=1:3, pch=8, cex=2)
```

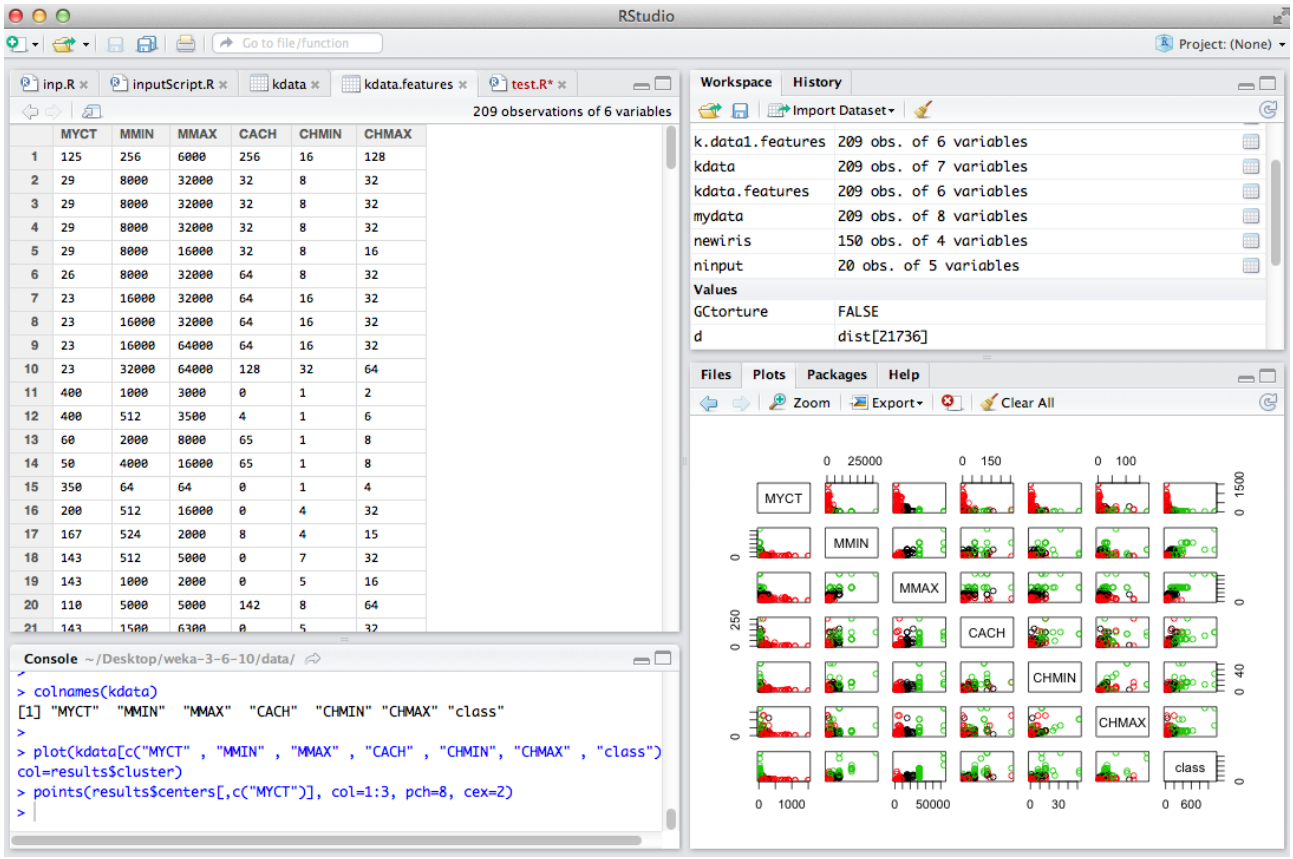
The code to the left can be explained as follows:

First, read the dataset and view for testing purpose. Second, access the dataset features the features built function within the library.

Third, apply the k-means function to the dataset. In this case, we are using 3 as the number of cluster and kdata is our dataset.

Fourth, display the dataset size, cluster, and center for testing purpose. Then, create a table with the classes and as many number of iteration as clusters.

Finally, plot the resulting dataset. You can use colnames to figure out the number and or name of columns, if countable, and plot them directly as shown below.

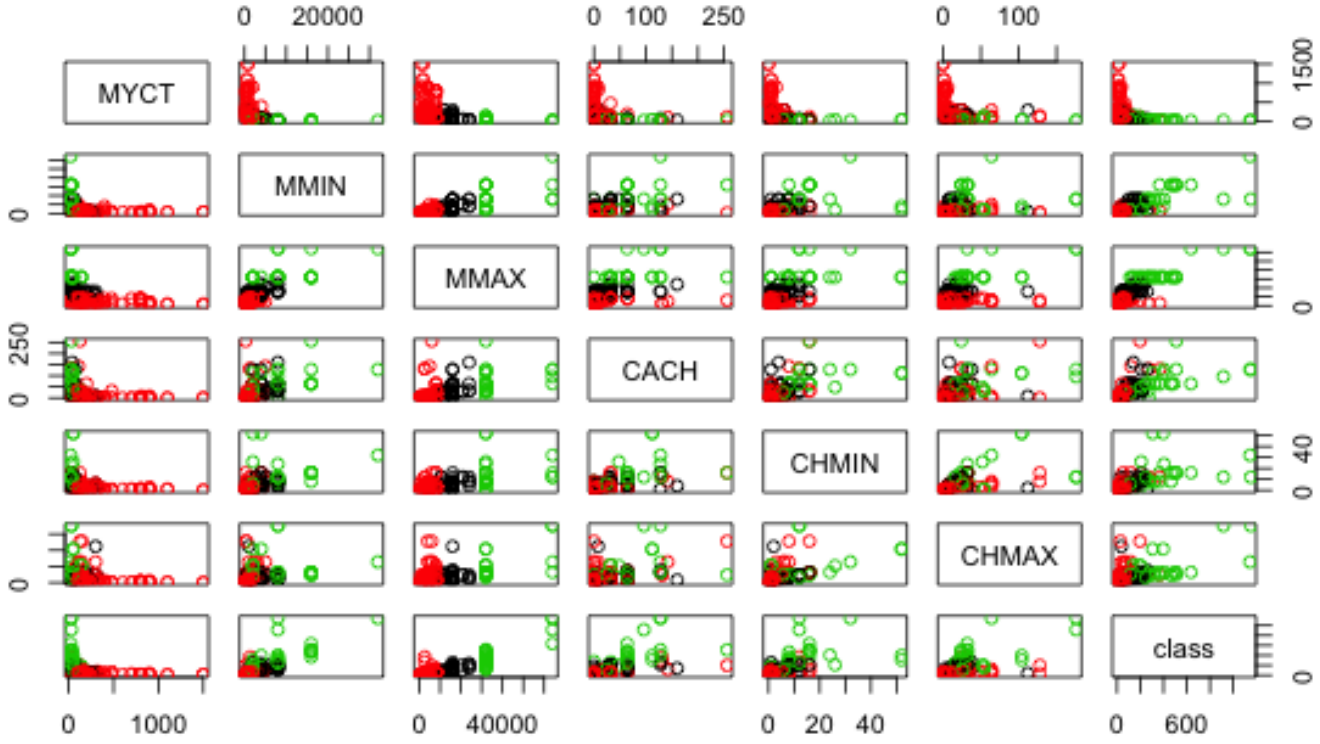


```
K-means clustering with 3 clusters of sizes 53, 129, 27

Cluster means:
       MYCT     MMIN      MMAX     CACH     CHMIN    CHMAX
1  82.22642 3493.434 15828.302 32.18868  4.773585 16.03774
2 287.17054 1226.791  4918.574 12.17054  2.558140 12.91473
3  44.29630 9481.481 36740.741 73.77778 14.777778 48.22222

Clustering vector:
  [1] 2 3 3 3 1 3 3 3 3 3 2 2 2 1 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1
 [36] 1 2 2 2 2 2 2 1 1 1 1 2 2 2 2 2 2 2 1 2 2 1 2 2 2 2 2 2 2 2 1 1 2
 [71] 2 2 2 2 2 2 1 2 1 2 1 2 2 1 2 2 2 2 2 3 2 3 3 2 1 1 3 3 3 1 2 2 2
[106] 2 2 2 2 2 2 2 2 2 2 1 1 2 1 1 1 1 2 2 2 2 2 2 1 1 1 2 2 2 1 1 1 2
[141] 2 2 2 1 1 1 1 1 1 2 1 3 3 3 1 3 3 2 2 2 2 2 2 1 1 2 2 1 1 1 2 2 2
[176] 1 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 3 3 2 3 3 3 3 3 2 2 2 2 2 2 2 2

Within cluster sum of squares by cluster:
[1]  797545559  947382443 4686079632
 (between_SS / total_SS =  79.7 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"
[5] "tot.withinss" "betweenss"    "size"         "iter"
[9] "ifault"
```

If you see in the previous image, it shows that our dataset has 209 observa-tions of 7 variables. Then, we proceeded to get the features of our dataset. Here, you can see that we have 209 observations and only 6 variables.

The image below is a visual representation of the clustering process result. It is very easy to distinguish the three different type of cluster within the dataset since each one of them has a different color.

Below is the output resulting from the above code, and this can be described in the following manner:



As we see in the first image above we have three clusters of different size. The first cluster is of size 53, the second cluster is of size 129, and the third cluster is of size 27. Next, it is the mean of each cluster's variable. The clustering vector tells me what cluster each observation is in. Next, is the within cluster sum of squares of each cluster. As we can see, the total percentage the sum of squares between over to total sum of squares is 79.7

We can see in the image, one cluster is in color green and the others are in color black and red. We can see that his grouping found a pretty good group between these different observations in the variables. Also, if observe the majority of the performance of the cpu is being used by the cluster 2, which is almost three times bigger than the other clusters.

```r
mydata <- read.arff("cpu.arff")
mydata <- na.omit(mydata) # listwise deletion of missing
mydata <- scale(mydata) # standardize variables
# Determine number of clusters
wss <- (nrow(mydata)-1)*sum(apply(mydata,2,var))
for (i in 2:15) wss[i] <- sum(kmeans(mydata, centers=i)$withinss)
plot(1:15, wss, type="b", xlab="Number of Clusters", ylab="Within groups sum of squares")

# K-Means Cluster Analysis
fit <- kmeans(mydata, 5) # 5 cluster solution
# get cluster means
aggregate(mydata,by=list(fit$cluster),FUN=mean)
# append cluster assignment
mydata <- data.frame(mydata, fit$cluster)

# Ward Hierarchical Clustering
d <- dist(mydata, method = "euclidean") # distance matrix
fit <- hclust(d, method="ward")
plot(fit) # display dendogram
groups <- cutree(fit, k=5) # cut tree into 5 clusters
# draw dendogram with red borders around the 5 clusters
rect.hclust(fit, k=5, border="red")

# Ward Hierarchical Clustering with Bootstrapped p values
fit <- pvclust(mydata, method.hclust="ward", method.dist="euclidean")
plot(fit) # dendogram with p values
# add rectangles around groups highly supported by the data
pvrect(fit, alpha=.95)

# Model Based Clustering
fit <- Mclust(mydata)
plot(fit) # plot results
summary(fit) # display the best model
```

The code to the left is another code that we did to further analyze the clustering process in R.

Prior to clustering data, we estimated missing data and rescaled variables for comparability.

K-mean requires the analyst to specify the number of clusters to extract. Plotting the within groups by number of clusters extracted helped to determine the appropriate number of clusters.

```
> table(kdata$class, results$cluster)
```

	1	2	3
6	2	0	0
7	1	0	0
8	1	0	0
10	1	0	0
11	3	0	0
12	4	0	0
13	1	0	0
14	2	0	0
16	5	0	0
17	2	0	0
18	4	0	0
19	1	0	0
20	4	0	0
21	2	0	0
22	6	0	0
23	1	0	0
24	5	0	0
25	2	0	0
26	4	0	0
27	3	0	0
28	1	0	0
29	2	0	0
30	4	0	0
31	1	0	0
32	6	1	0
33	3	0	0
34	2	0	0
35	1	1	0
36	5	0	0
37	1	0	0
38	3	1	0

	1	2	3
38	3	1	0
40	5	1	0
41	1	0	0
42	1	1	0
44	1	0	0
45	4	1	0
46	1	1	0
49	0	1	0
50	4	2	0
51	1	0	0
52	1	1	0
53	0	1	0
54	0	1	0
56	1	0	0
58	1	0	0
60	2	2	0
61	1	0	0
62	3	0	0
63	0	1	0
64	1	0	0
65	0	1	0
66	1	3	0
67	1	0	0
69	1	0	0
70	1	1	0
71	2	0	0
72	1	1	0
74	0	1	0
75	0	1	0
76	1	1	0
77	1	0	0
80	0	1	0
84	0	1	0
86	0	1	0

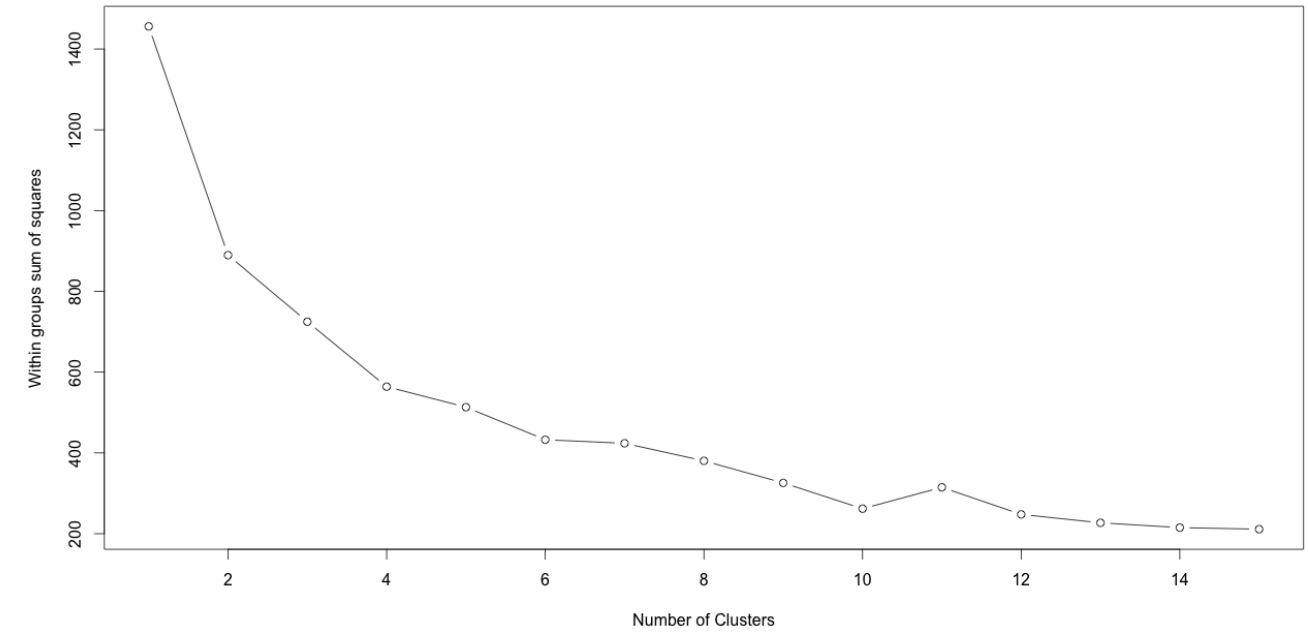
	1	2	3
92	1	0	0
93	0	1	0
100	0	2	0
105	1	0	0
106	0	1	0
109	1	0	0
111	0	1	0
113	0	1	0
114	0	0	1
116	1	0	0
120	1	0	0
130	0	1	0
132	0	2	0
133	0	1	0
134	0	0	1
136	0	1	0
138	0	2	0
140	0	1	0
141	0	0	1
143	0	1	0
144	1	1	0
172	0	0	1
173	0	1	0
185	0	1	0
188	0	1	0
189	0	0	1
198	1	0	0
208	0	1	1
212	0	1	0
214	0	2	0
220	0	0	1
237	0	1	0
248	0	0	1
259	0	1	0
269	0	0	1
274	1	0	0
277	0	0	2

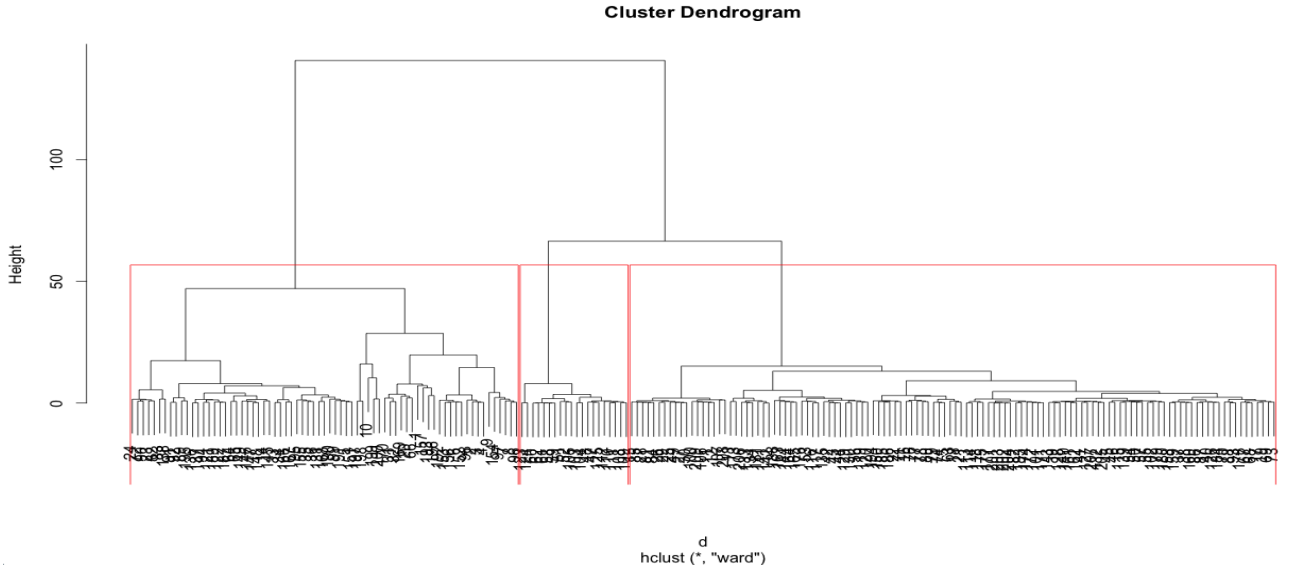
	1	2	3
144	1	1	0
172	0	0	1
173	0	1	0
185	0	1	0
188	0	1	0
189	0	0	1
198	1	0	0
208	0	1	1
212	0	1	0
214	0	2	0
220	0	0	1
237	0	1	0
248	0	0	1
259	0	1	0
269	0	0	1
274	1	0	0
277	0	0	2
307	0	0	1
318	0	0	1
326	0	0	1
367	0	0	1
368	1	0	0
370	0	0	1
397	0	0	1
405	0	0	1
465	0	0	2
489	0	0	1
510	0	0	2
636	0	0	1
915	0	0	1
1144	0	0	1
1150	0	0	1

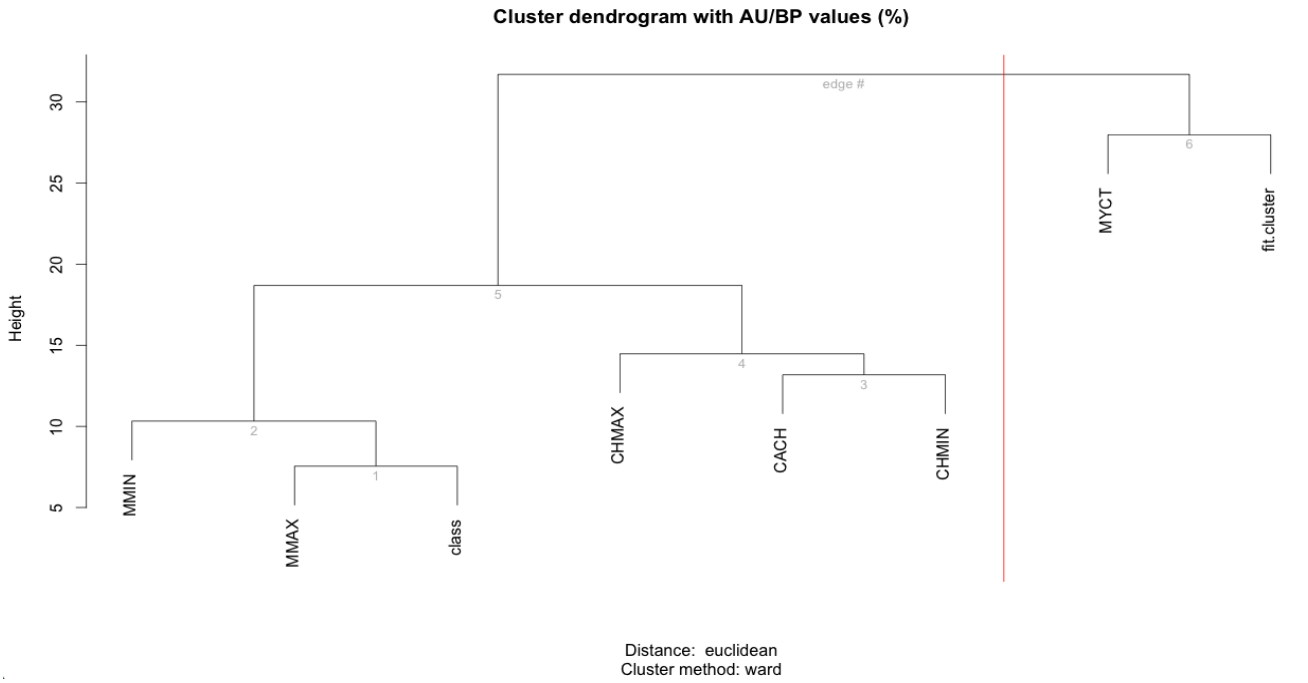
The table to the left is explaining in number how this grouping actually did, in other words how clustering did compared to the real stuff. We can see in the table how many of these classes match up with the resulting clusters that we got.

What we have to the left, for example, in line 6. This row has 2 observations and every single one of these observations was found in cluster one. In cluster one there are multiple observations as we can see in the table, cluster 2 has no observation till line 30, and cluster 3 has no observation till line 113.

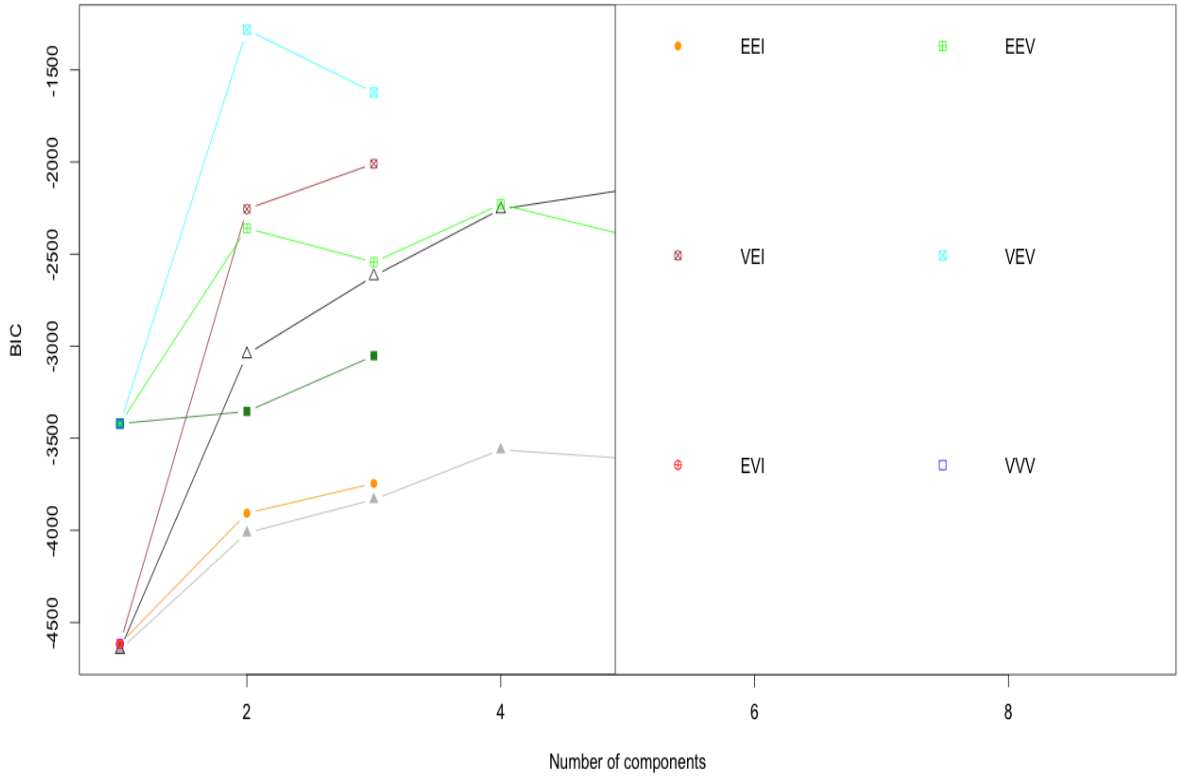
On the other hand, we can see overlaps in lines 32, 35, 38, 40, 42, 45, 50, 52, 60, 66, 70, 72, 76, 144 and 208, where different classes have part of a specific group that is being observed.

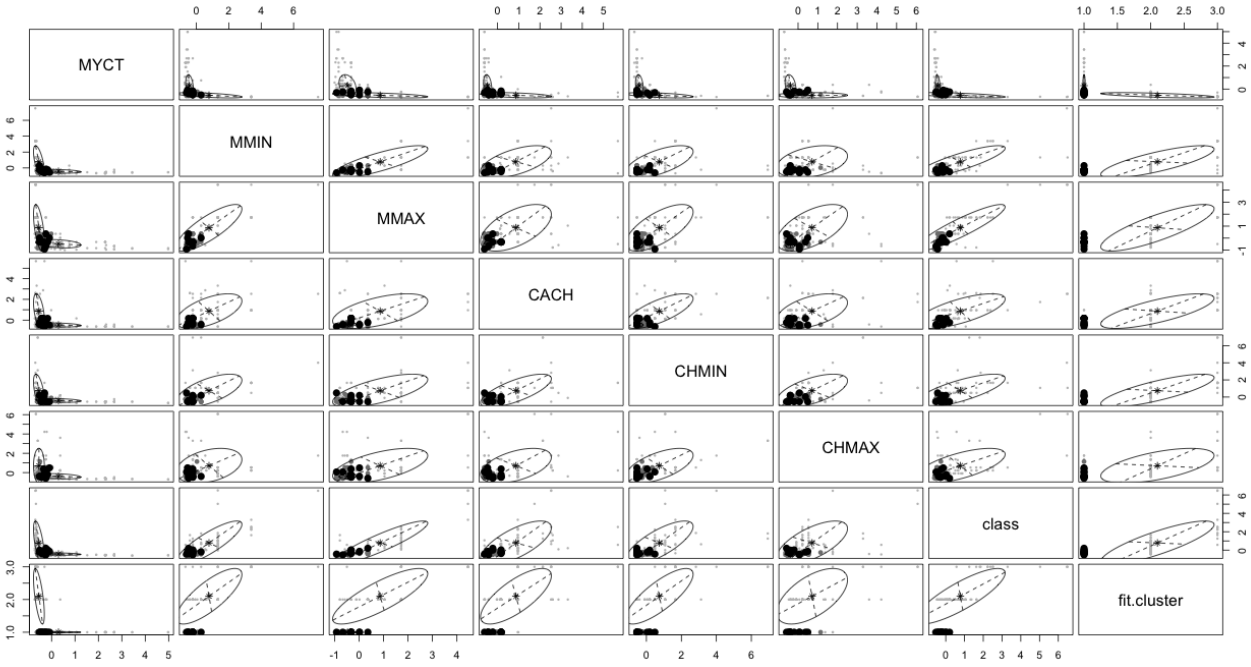






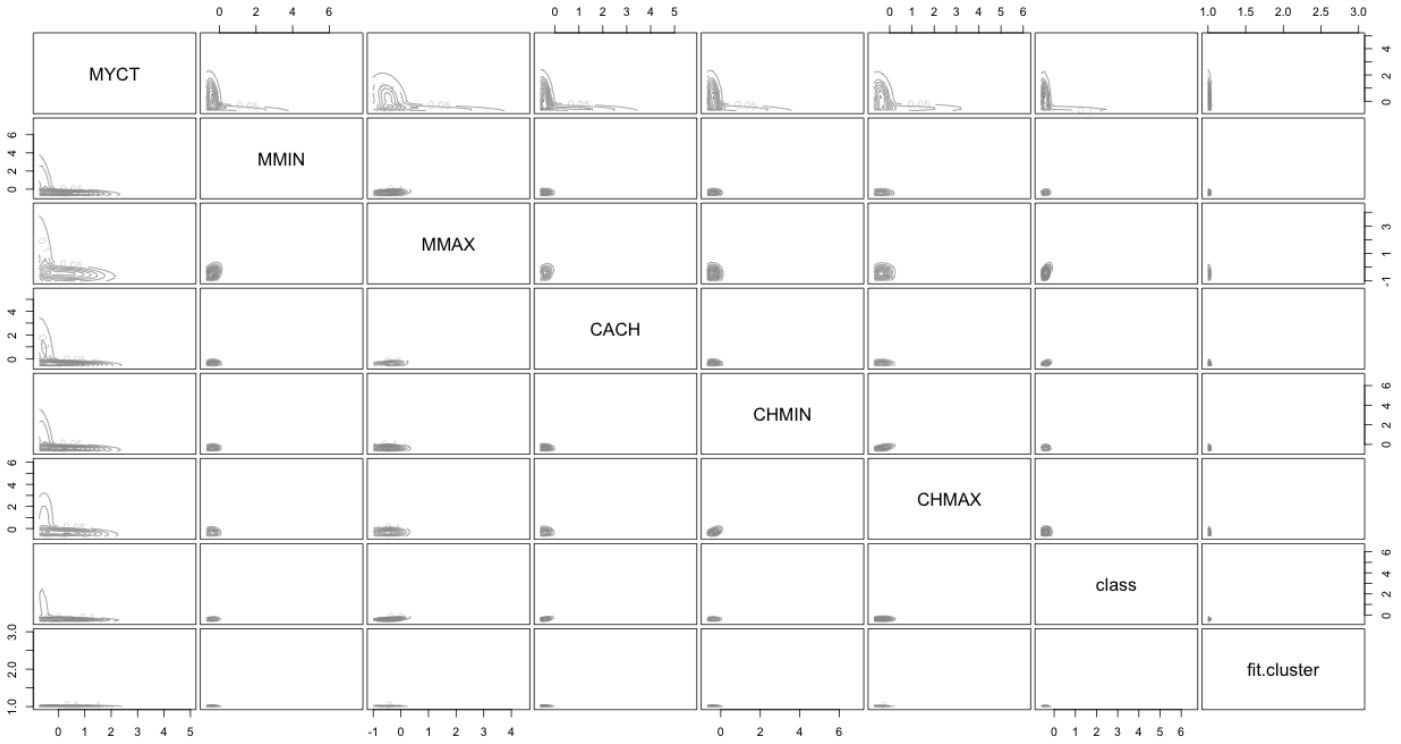
The above is the result of the hierarchical clustering approach that we used. This function provides the p-values for hierarchical clustering in our dataset based on multi-scale bootstrap resampling. As we can see in the image above, clusters that are highly supported by the data have large p values.

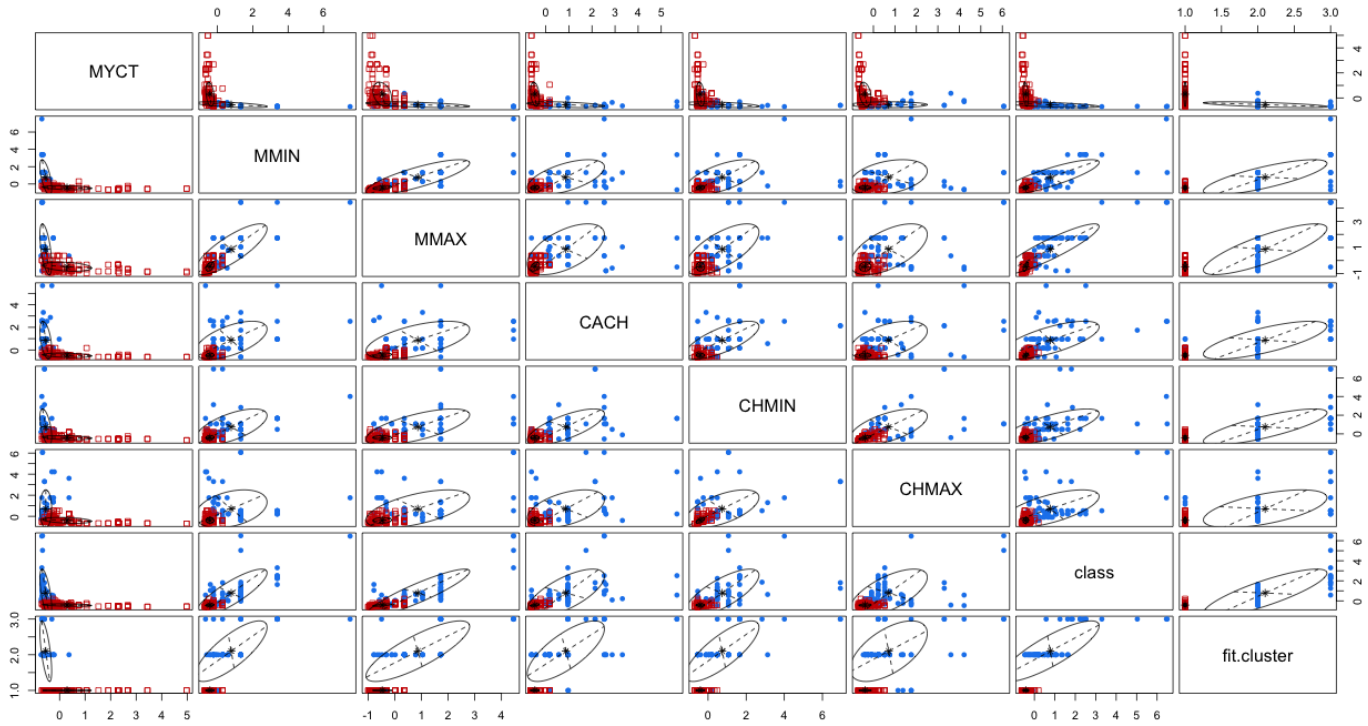




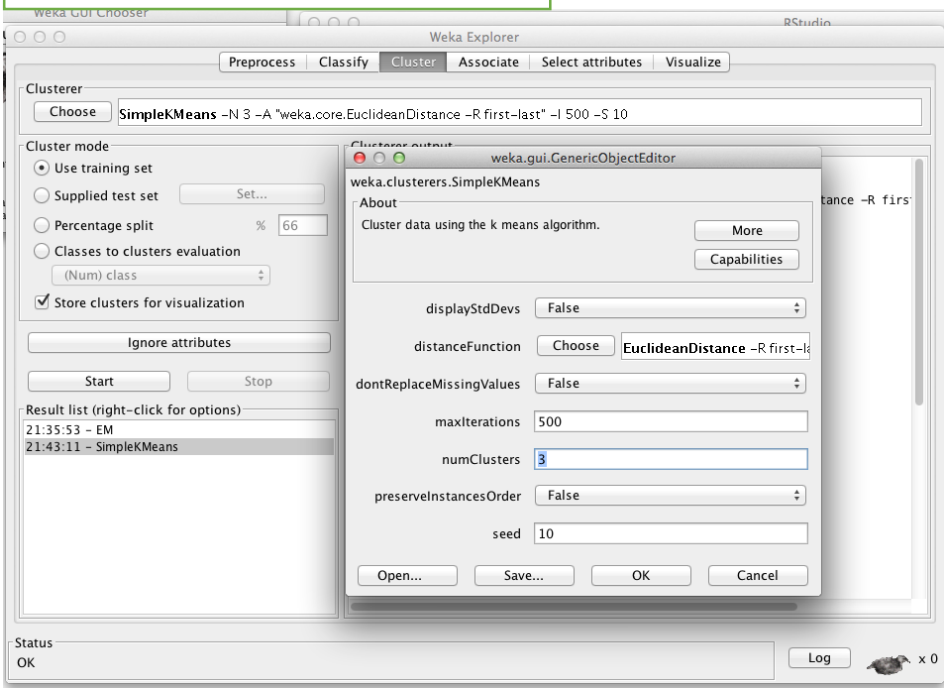

```
> aggregate(mydata,by=list(fit$cluster),FUN=mean)
  Group.1       MYCT       MMIN       MMAX       CACH       CHMIN       CHMAX      class
1       1  0.2703425 -0.4140540 -0.4654160 -0.4460203 -0.3979666 -0.3332001 -0.4149374
2       2 -0.5553613  0.4570097  0.6277376  0.5861156  0.4068401  0.3641878  0.3515982
3       3 -0.6396064  2.4695828  2.3442127  2.3050541  2.4963378  2.0008899  2.8776003
```

This other approach that we used assume a variety of data models and apply maximum likelihood estimation and Bayes criteria to identify the most likely model and number of clusters. As we can see in the top left image, the optional models are selected to BIC for EM initialized by hierarchical clustering for parameterized Gaussian mixture models. Here, the model and number of clusters with larges BIC is chosen. The other images give a very nice and organized interpretation of the cluster processing result. You can easily identify the groups within each section and determine the overlaps according with the values that are displayed in the confusion matrix.


MYCT
MMIN
MMAX
CACH
CHMIN
CHMAX
class
fit.cluster


MYCT
MMIN
MMAX
CACH
CHMIN
CHMAX
class
fit.cluster

b) Weka Clustering Implementation



To perform clustering, select the cluster tab in the Explorer and click choose button. This results in a drop down list of variables clustering algorithms. In this case we select SimpleKMeans. Next, click on the text box to the right of the choose button to get the pop-up window shown in the figure in the top left of this page, for editing the clustering parameter. In the pop-up window we entered 3 to be the number of cluster, the same as the one we used in the R cluster implementation approach. The seed value is used to generating a random number which is, in turn, used for making the initial assignnnnment of instances of clusters. In general, K-mean is quite sensitive to how clusters are initially assigned. Thus, it is often necessary to try different values and evaluate results.

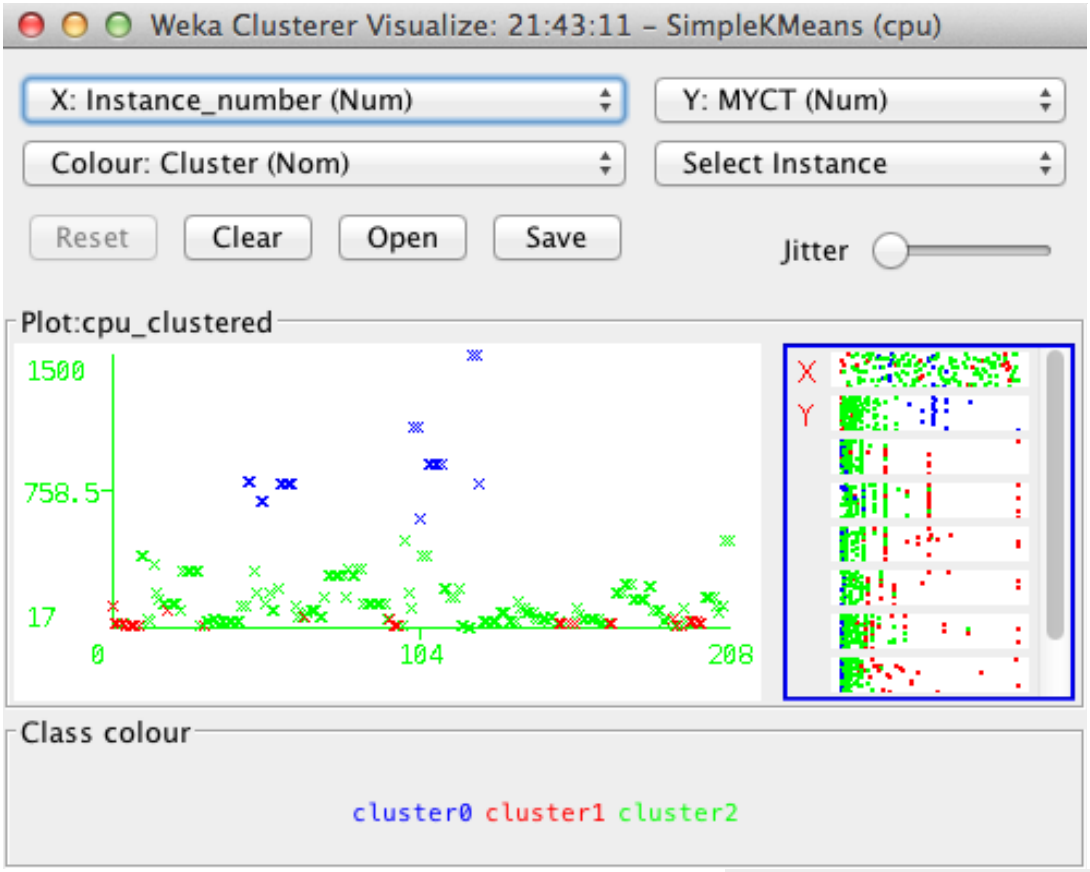
Note: The WEKA SimpleKMeans algori-thm uses Euclidean dist-ance measure to compute distances between instances and clusters.

The result window shows the centroid of each cluster as well as statistics on the number and percentages of instances assigned to different clusters.

Clusterer output

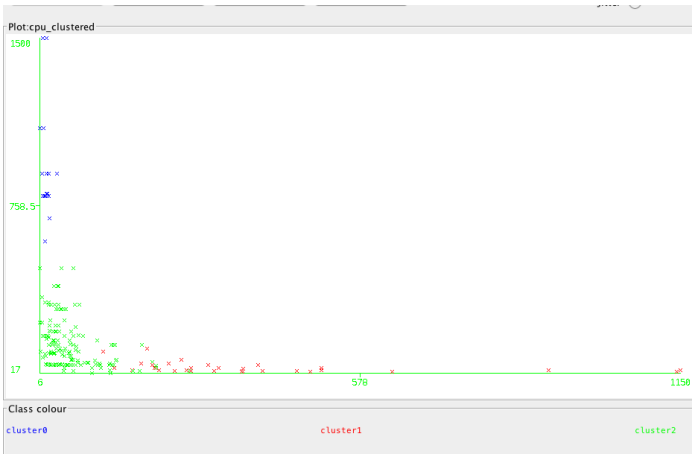
```
kMeans
======

Number of iterations: 10
Within cluster sum of squared errors: 16.2226477893351
Missing values globally replaced with mean/mode

Cluster centroids:
                         Cluster#
Attribute    Full Data          0          1          2
                 (209)       (20)       (31)      (158)
=============================================================
MYCT            203.823        906    40.4194        147
MMIN           2867.9809     664.8  9050.5161  1933.8354
MMAX          11796.1531    4000.6 31407.0968  8935.2152
CACH             25.2057         1         95    14.5759
CHMIN             4.6986      0.95    15.1935     3.1139
CHMAX            18.2679      2.05    47.8065    14.5253
class            105.622     17.75   392.4839     60.462



Time taken to build model (full training data) : 0.01 seconds

=== Model and evaluation on training set ===

Clustered Instances

0       20 ( 10%)
1       31 ( 15%)
2      158 ( 76%)
```

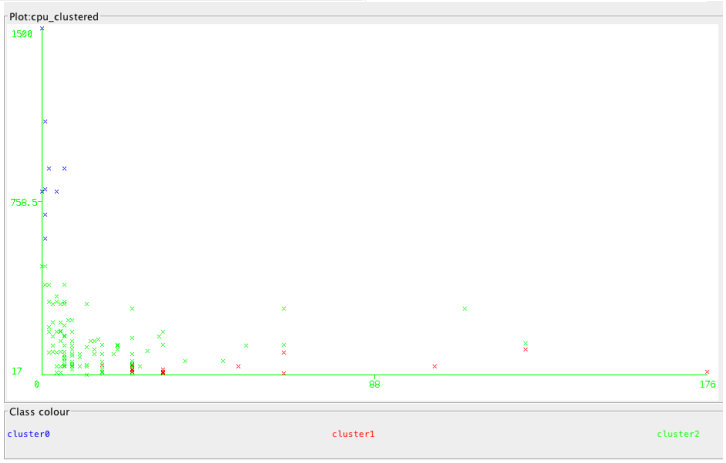
Cluster centroids are the mean vector for each cluster (so, each dimension value in the centroid represents the mean value for that dimension in the cluster). This, centroids can be used to characterize the cluster. For example, the centroid for cluster. For example, cluster 2 is 76% which is more than three times bigger than the others. This means that cluster 2 is using the CPU performance way faster than the other two clusters.

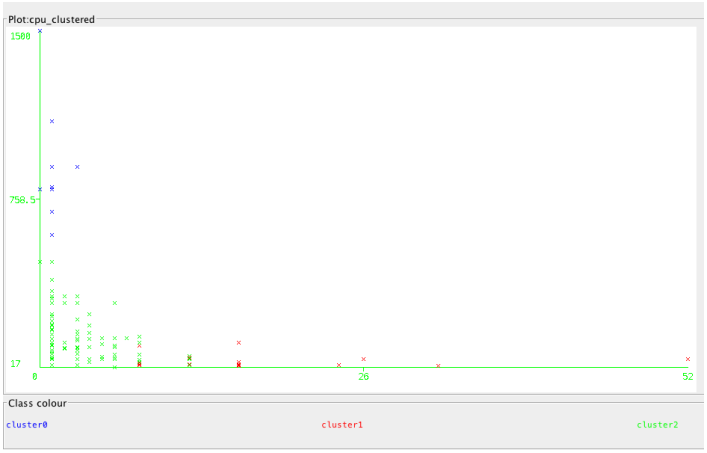


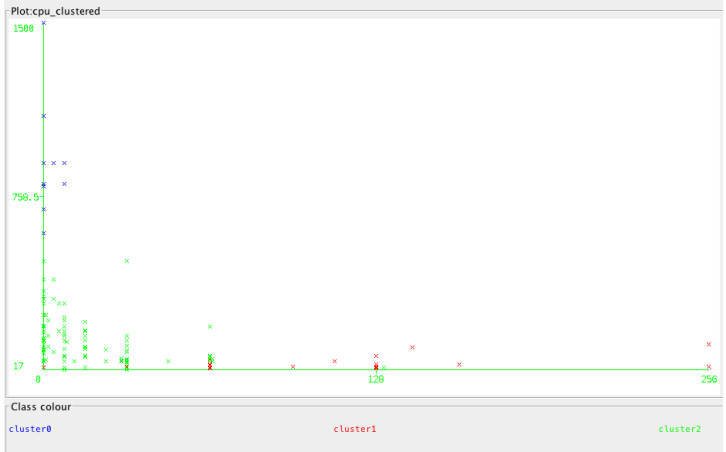
This is another way of understanding the characteristic of each cluster through visualization. Here, you can choose the cluster number and any of the other attributes for each of the three different dimensions available. Different combinations of choices will result in a visual rendering of different relationships within each cluster.

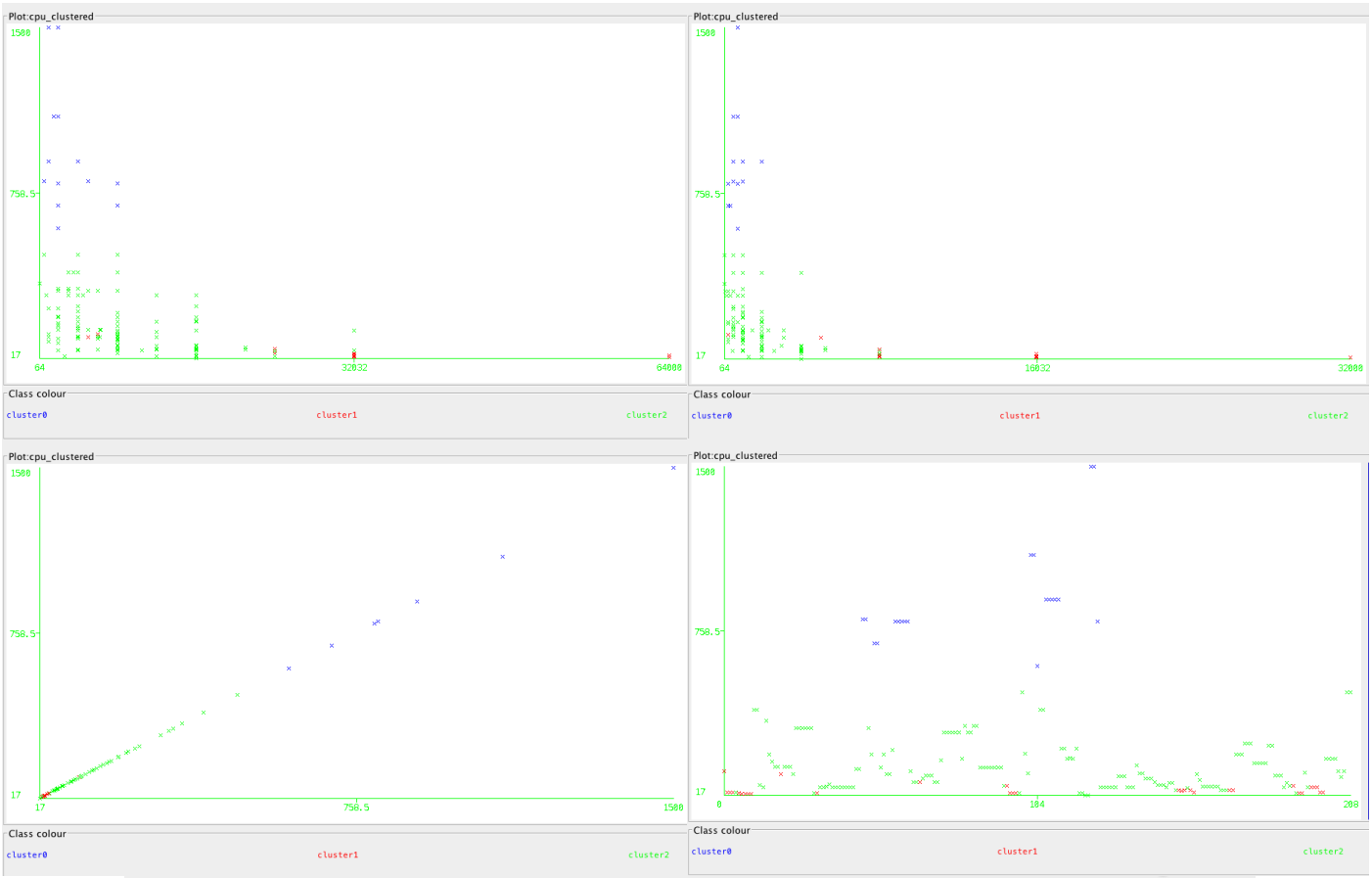
You can see that cluster 2 and 1 are dominated, and therefore use the cpu performance the most, while cluster 1 uses the least of the cpu performance

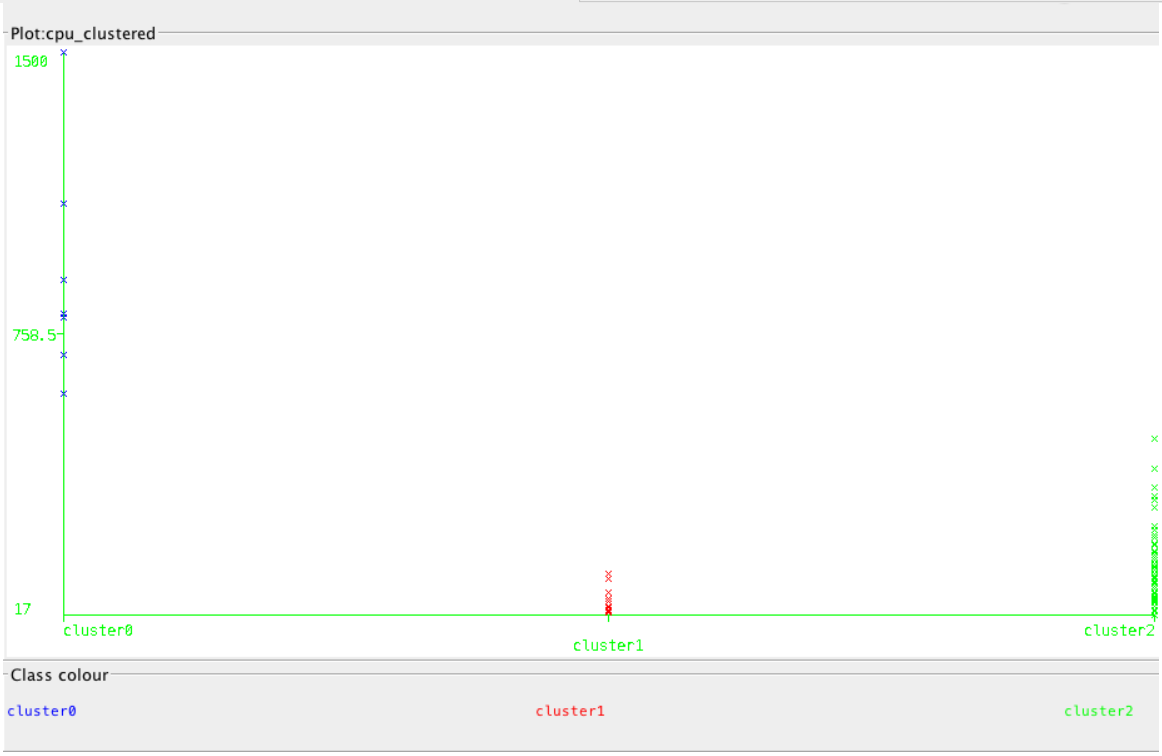












These all images are just for a better interpretation of the three clusters. This graphically shows how WEKA clustering process is done.

WEKA VS R CLUSTER IMPLEMENTATION:

I personally would choose R cluster implementation over WEKA's based on the results obtained above. In WEKA we were able to see EM and K-model, as well as the XMeans, hierarchical, and optics tools, which were very helpful to interpret the dataset in terms of its three clusters along with its farthest first clustering. In R were also able to see the EM and K-model, as well as other tools such as hierarchical clustering, agglomerative nesting, Fuzzy C-means clustering, bagged clustering, Cluster ensembles and convex clustering. This tools allowed to process the cluster analysis further and better distinguish and capture the confusion between the clusters within a matrix.